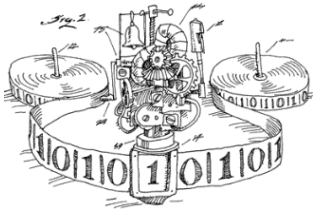
Teoría de la Computación y Lenguajes Formales



Teoría de la Computación y Lenguajes Formales: Sesión 6. Lenguajes Regulares y Expresiones Regulares

Édgar Chacón Ramírez
Departamento de Computación
Semestre 2026 - B

Alcance de la sesión 6-B

UNIVERSIDAD DE LOS ANDES VENEZUELA

Minimización por eliminación de conjuntos

Actividad

Minimización de autómatas. Algoritmo 2

UNIVERSIDAD DE LOS ANDES VENEZUELA

Bases

Particionar los estados del autómata en grupos disjuntos de estados equivalentes.
Principio:

1. Dos estados son equivalentes si aceptan las mismas cadenas.

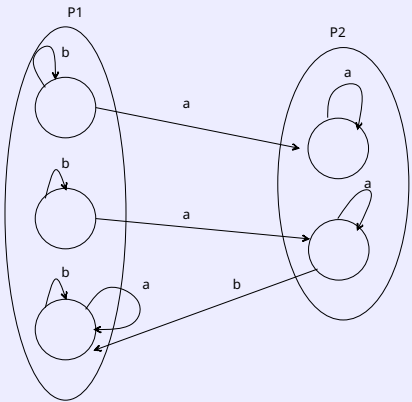
$$w \in \Sigma^*, f_t(q_0, w) \in F \iff f_t(q_1, w) \in F$$

Procedimiento:

1. Asumir dos particiones iniciales posiblemente equivalentes
   F y $Q \setminus F$
2. Analizar cada partición mediante la revisión de las transiciones de cada estado dentro de la partición. Si los estados dentro de la partición llevan en cada símbolo a las mismas particiones son equivalentes. En caso contrario separar las particiones en nuevas particiones

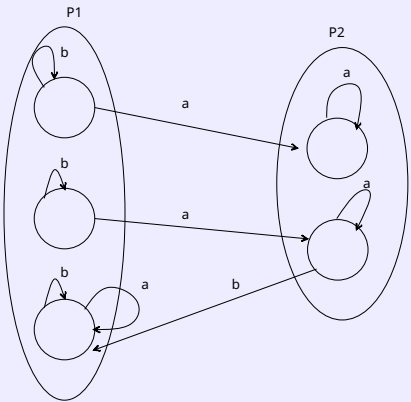
Minimización de autómatas. Algoritmo 2

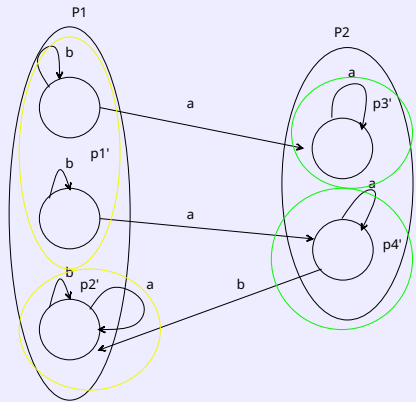
Ejemplo



Minimización de autómatas. Algoritmo 2

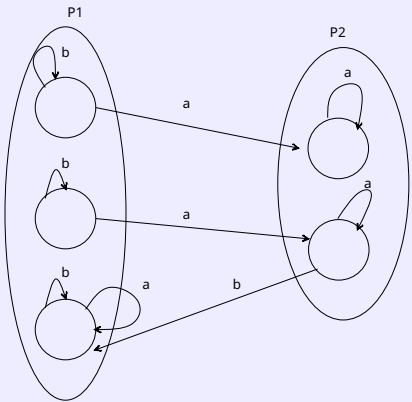
Ejemplo





Minimización de autómatas. Algoritmo 2

Ejemplo

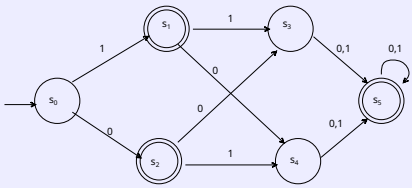


El algoritmo finaliza al no existir cambios en la siguiente iteración

Minimización de autómatas. Algoritmo 2

Aplicándolo en el mismo autómata

Minimizar el AFD siguiente



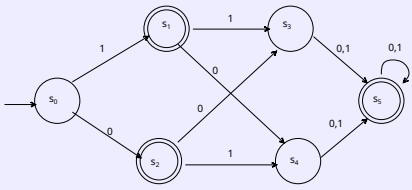
$P_0 = \{s_1, s_2, s_5\}$
$P_1 = \{s_0, s_3, s_4\}$

		0	1	
P_0	s_1	s_4 P_1	s_3 P_1	P_2
P_0	s_2	s_3 P_1	s_4 P_1	P_2
P_0	s_5	s_5 P_0	s_5 P_0	P_3
P_1	s_0	s_2 P_0	s_1 P_0	P_1
P_1	s_3	s_5 P_0	s_5 P_0	P_1
P_1	s_4	s_5 P_0	s_5 P_0	P_1

Minimización de autómatas. Algoritmo 2

Aplicándolo en el mismo autómata

Iteración 2



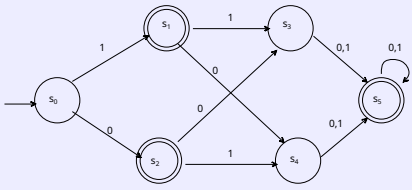
$P_2 = \{s_1, s_2\}$
$P_3 = \{s_5\}$
$P_1 = \{s_0, s_3, s_4\}$

	0	1	
P_2 s_1	s_4 P_1	s_3 P_1	P_2
P_2 s_2	s_3 P_1	s_4 P_1	P_2
P_3 s_5	s_5 P_3	s_5 P_3	P_3
P_1 s_0	s_2 P_2	s_1 P_2	P_4
P_1 s_3	s_5 P_3	s_5 P_3	P_5
P_1 s_4	s_5 P_3	s_5 P_3	P_5

Minimización de autómatas. Algoritmo 2

Aplicándolo en el mismo autómata

Iteración 3



$P_2 = \{s_1, s_2\}$
$P_3 = \{s_5\}$
$P_4 = \{s_0\}$
$P_5 = \{s_3, s_4\}$

	0	1	
P_2 s_1	s_4 P_5	s_3 P_5	P_2
P_2 s_2	s_3 P_5	s_4 P_5	P_2
P_3 s_5	s_5 P_3	s_5 P_3	P_3
P_4 s_0	s_2 P_2	s_1 P_2	P_4
P_5 s_3	s_5 P_3	s_5 P_3	P_5
P_5 s_4	s_5 P_3	s_5 P_3	P_5

No hay cambio

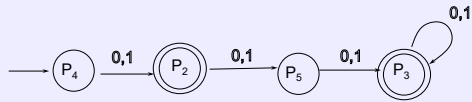
Minimización de autómatas. Algoritmo 2

Resultado

Iteración 3

	0	1	
P_2 s_1	s_4 P_5	s_3 P_5	P_2
P_2 s_2	s_3 P_5	s_4 P_5	P_2
P_3 s_5	s_5 P_3	s_5 P_3	P_3
P_4 s_0	s_2 P_2	s_1 P_2	P_4
P_5 s_3	s_5 P_3	s_5 P_3	P_5
P_5 s_4	s_5 P_3	s_5 P_3	P_5

Autómata resultante



Minimización de autómatas. Algoritmo 2

UNIVERSIDAD DE LOS ANDES VENEZUELA

Paso 1

Creación de dos particiones iniciales. Estados terminales y no terminales

Creación de las dos particiones iniciales

```
26
27 # Crear primeras particiones
28 grupos=list()
29 for est in q:
30     if est in f:
31         grupos.append([est,1])
32     else:
33         grupos.append([est,2])
34 cant_grupos = 2
35 print('grupos', grupos)
```

Minimización de autómatas. Algoritmo 2

UNIVERSIDAD DE LOS ANDES VENEZUELA

Paso 2

Refinación de las particiones.

Creación de una tabla para cada partición y verificación del destino de cada estado en la partición

```
39    ncant_grupos = 0
40    tabla = list()
41    icont=0
42    for est in q:
43        linea = [est,grupos[icont][1]]
44        icont=icont+1
45        for sim in s:
46            gruposal = 0
47            for trans in transiciones:
48                if( (trans[0]==est ) and (trans[1] == sim)):
49                    estsal = trans[2]
50                    for grupo in grupos:
51                        if estsal == grupo[0]:
52                            gruposal = grupo[1]
53            linea.append(gruposal)
54        tabla.append(linea)
55
```

Minimización de autómatas. Algoritmo 2

UNIVERSIDAD DE LOS ANDES VENEZUELA

Paso 3

Refinación de las particiones.

Definición de las nuevas particiones. Almacenar en una nueva tabla de particiones

```
57    # analisis de evolucion para definir nuevas particiones
58    nrogrupnv =0
59    gruponuevo = list()
60    for qest in q:
61        gruponuevo.append([qest,0])
62    #analisis de cada grupo existente de la iteración anterior
63    for nrogrupo in range(1,cant_grupos+1):
64        #creacion de tablas de analisis. Una tabla para cada grupo
65        tabla_analisis = list()
66        for linea in tabla:
67            if linea[1] == nrogrupo:
68                lineaan = linea
69                lineaan.append('')
70                tabla_analisis.append(lineaan)
71            np = list()
72            for lineaan in tabla_analisis:
73                cadena = ''
74                for i in range(2,len(lineaan)-1):
75                    if isinstance(lineaan[i],int):
76                        cadena = cadena + str(lineaan[i])
77                lineaan[len(lineaan)-1] = cadena
78                if cadena not in np:
79                    np.append(cadena)
80        # encontrar los grupos nuevos al no ser equivalentes
81        for cadena in np:
82            nrogrupnv = nrogrupnv+1
83            for lingruponuevo in gruponuevo:
84                for linea in tabla_analisis:
85
86                    if (lingruponuevo[0] == linea[0]) and (linea[len(linea)-1] == cadena):
87                        lingruponuevo[1]=nrogrupnv
88
```

Minimización de autómatas. Algoritmo 2

UNIVERSIDAD DE LOS ANDES VENEZUELA

Paso 4

Comprobación de la finalización del procedimiento

Comparar la tabla de partición anterior con la nueva tabla. Si diferentes aplicar de nuevo el procedimiento

```
90    # comparación con la iteración anterior
91    iguales = True
92    for i in range(len(q)):
93        if(gruponuevo[i][1] != grupos[i][1]):
94            iguales = False
95            break
96    iter = iter+1
97    if not iguales:
98        grupos = gruponuevo
99        cant_grupos = ncant_grupos
100   else:
101       break
102   if iter == 50:
103       break
104
```

Minimización de autómatas. Algoritmo 2

Corrida de un ejemplo

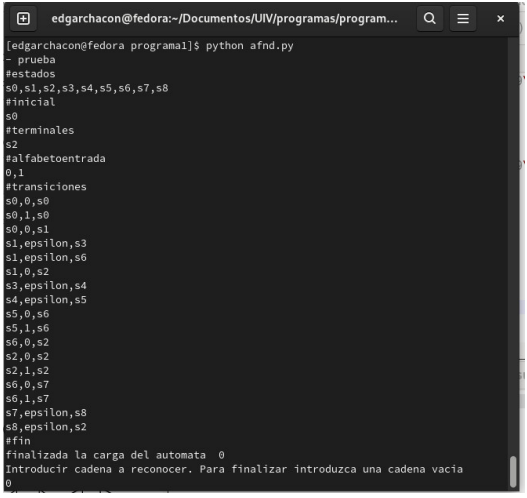
Autómata ejemplo

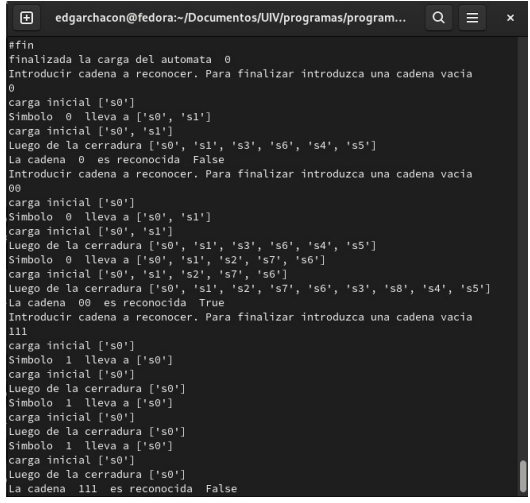
```
q = ['s0','s1','s2','s3','s4','s5','s6','s7']
f = ['s2']
sin = 's0'
s = ['0','1']
transiciones = list()
#transiciones
transiciones.append(['s0','0','s1'])
transiciones.append(['s0','1','s5'])
transiciones.append(['s1','0','s6'])
transiciones.append(['s1','1','s2'])
transiciones.append(['s2','0','s0'])
transiciones.append(['s2','1','s2'])
transiciones.append(['s3','0','s2'])
transiciones.append(['s3','1','s6'])
transiciones.append(['s4','0','s7'])
transiciones.append(['s4','1','s5'])
transiciones.append(['s5','0','s2'])
transiciones.append(['s5','1','s6'])
transiciones.append(['s6','0','s6'])
transiciones.append(['s6','1','s4'])
transiciones.append(['s7','0','s6'])
transiciones.append(['s7','1','s2'])
```

```
edgarchacon@fedora:~/Documentos/UIV/programas/algoritm...
[edgarchacon@fedora algoritmoMinimizacion]$ python creararreglo.py
grupos [['s0', 2], ['s1', 2], ['s2', 1], ['s3', 2], ['s4', 2], ['s5', 2], ['s6',
 2], ['s7', 2]]
minimizacion del automata en  3  iteraciones
estado en automata  s0  particion  2
estado en automata  s1  particion  4
estado en automata  s2  particion  1
estado en automata  s3  particion  5
estado en automata  s4  particion  2
estado en automata  s5  particion  5
estado en automata  s6  particion  3
estado en automata  s7  particion  4
[edgarchacon@fedora algoritmoMinimizacion]$
```

Autómata de prueba

UNIVERSIDAD DE LOS ANDES VENEZUELA

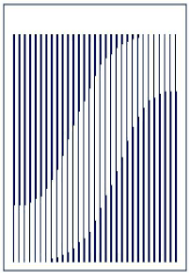




¡Muchas gracias!



UNIVERSIDAD
DE LOS ANDES
VENEZUELA



Édgar Chacón Ramírez
email: echacon@ula.ve